ActInf Livestream 055.2 ~ "Realising Synthetic Active Inference Agents" Pa

Livestream | Part I & Part II | 1:57:18

Hello and welcome everyone. It's November 16th, 2023. We're in ActInf Livestream 55.2
and it's our second discussion with Magnus here on the message passing papers. In this .2,
we're going to begin with some code explorations, see where it goes, and then in the last section
have more open discussion and see where that goes. So thank you for joining again, Magnus,
and looking forward to seeing this code. Yeah, thanks for having me. So yeah, one of the things
we talked about last time was that it would be nice to go through what all of this theory actually
looks like in code. So I brought along something that I prepared earlier, which is basically
the experiments that go with the papers. And I think the one that makes the most sense to look at
is the one where we do direct policy inference, because that's sort of one of the really novel
things where we don't encapsulate prior algorithms. We actually get something new. So that is what we
hopefully have on screen right now. All of this is implemented in Julia and it is all publicly
available. So for anyone who's interested, you can go and mess around with this on your own if you
want. But basically, the model we're going to be looking at is the same discrete POMDP that we all
know and love, but instead of computing expected free integers and comparing them to find our policy,
policy. We're just going to infer the policy in one go as part of optimization.
So would it make sense if I just go through it cell by cell or?
Yeah. Maybe make the font a little bigger and then go cell by cell.
Yeah. Perfect. Thank you.
All right. Is this better?
Yeah.
Sweet. So all this is in Julia and it runs out of notebooks. There's instructions for how to do it
and the repo. So the first thing we do is we just load a bunch of things that we need.
These here are all message passing rules for the transition mixture model.
The transition mixture model is what allows us to do policy inference. It's basically saying
instead of having one transition matrix, one B matrix, we have a number of candidate ones
and then we have a variable that just picks one. So let's say I have either one transition matrix,
another transition matrix. If my indicator variable is 1, 0, I pick this one. If it is 0, 1, I pick this one.
That's sort of the logic here.
Then we have this goal observation node, which is a goal constrained observation node. And this is where we do
the GV based message parsing. So this is where we have the P substitutions going on that we talked about last time
and where we get to get all our epistemics. And then we have a bunch of helpful functions to set up
priors for stuff, whatever. So what this code implements is the teammates experiment.
And again, I think we all know how the teammates works. But just for completeness sake,

actually, I think I might be able to pull up the paper and show the graphic for the teammates because then we can see exactly what it is for modeling. We can see how that corresponds to the graph and everything. Just give me two seconds. I'll do that.

Here we go.

So this is the teammates environment. And this is the repo where you can find the code.

In the teammates environment, the agent starts in one of four positions, which we like to one.

And it's designed to simulate a rat looking for cheese in a maze.

This is a reward at either positions two or three. So I know one of these arms.

The main point is the agent doesn't know which arm has the reward at.

The one of these transitions. To learn that, it needs to go to position four. Position four has a queue that tells it whether the arms, the board is likely to be to the left or to the right.

So we need four transitions. We want to go to states one, two, three, and four.

And then we need a likelihood matrix that sort of maps to different observations that we can get.

Where are we at location? Do we get a reward? That kind of thing from all these four different locations.

So I think I actually have, yeah. So I can actually show the constraints FFG of the model that we're building.

And it looks like this.

So what's going on here is that we have a vector D that parameterizes where we start. This is this prior.

Then we have this transition mixture node.

The transition is from a latent state at time t to time t plus one.

It has four candidate transition matrices, again, because we need to move to either one, two, three, or four.

And then we have this indicator variable that we call U. And that just picks whether we go to one, two, three, or four.

Down here is where we have the, to me, the really interesting bit.

This is where we perform this P substitution that you can see here.

We have a mean field factorization and a P substitution.

And we have this composite node, this goal observation node with a goal prior.

So what this says is do GV based optimization around this node.

And the goal is given by this categorical down here.

So it's going to have some C vector that parameterizes it.

It's the same kind of notation you're going to find in most of the tables.

We're going to infer a two-step policy.

So we need to go from initial point, the ZT to t plus one to t plus two.

I think I should need to put two plus two in here, but no matter.

Does this model in this graph make sense?

Perfect.

The thing that I want to sort of point out is that we can see all these things,

where we do the EFB based things, the kind of messages that we need to pass,

effectorizations, all of this is something that we can see from the graph,

because there's a constrained FFG instead of a normal FFG.
So let's look at what this little guy looks like in code.
And it looks like this.
So we have a categorical with the D vector that starts.
We have C, which is going to be our goal priors.
We have our latent states.
And we have our indicator variables.
Each indicator variable gets a flat prior.
That's what this is.
Yeah.
So for all time steps, for all future time steps we want to plan over,
we need an indicator variable with a flat prior.
So we are a priori indifferent as to which state we want to go to.
We have a transition mixture that maps from
Z at the previous time step to Z at the current time step, Z.
And it takes us input the indicator variable, or switch,
and our candidate transition matrices.
We have the same thing.
We have the next node, which is this goal-constrained observation node.
It has a likelihood matrix, i.e. this A here.
And a goal priori, which is the C here.
C.
Input from the state.
And the likelihood matrix.
This here is some plumbing that we need to get the messages to work out.
So don't worry too much about this.
And this is actually the model specification.
This is all there is to it.
Which to me is kind of nice, because if we do inference in the back end,
then if I want to build models and design agents, all I really need should be able to should have to worry about
is to set up my model specification correctly.
All right.
All right.
All right.
This is another.
There's a little bit of plumbing.
And these are like optional constraints that we are going to introduce later.

What this basically says is that we want to point-mask constrain

The switch variables, the indicators.
So instead of saying, I'm sort of indifferent between going to two and three, I put equal
probability on them.
We're forcing the agent to make a choice.
So you can only pick one.
And this is identical to having a point-mask constraint or a delta constraint as described in part one.
That's what this thing does.
Right.
So then let's set up everything we need.
We have a planning horizon of two.
We want to run 10 inference iterations because this optimization procedure is an iterative thing.
And then we also need to set some initial marginals, like an BMP thing.
And we need to construct our A, B, C, and D matrices,
which for those of you that have worked with pi MDP or SPM or similar, these work the exact same way.
And then all we need to do is call the inference function
with the model with all our parameters.
So the team-based model that we just defined with our A, B,
the matrices, our D vectors to start with and our time horizon C.
We set our goal prior as data inputs,
which is just a convenience thing we did to make life a little easier, sort of programmatically.
And then we're off to the races.
So let's actually see what happens when we run this.
So if we just load everything,
should be up to date in a minute here.
So let's create a model.
Create our constraints.
Configure our experiment.
And now we are here, so we can run inference.
So do you have any guess as to how long this is going to take?
11 millifristians.
It's pretty accurate.
So it actually takes a little bit the first time because the model needs to be compiled.
Now you can see we have new posteriors available for our switch variable,
our state variables, and our initial state variable.
So let's see what we got.
So what we're printing here is just the distribution, the posterior marginal,

over the indicator variable, i.e. the probability of choosing go to 1, go to 2, go to 3, go to 4.
And what we get is this long number that we could round to about 25% chance of going to 4.
About 20% chance of going to either 2 or 3, and 35% chance of going to 4.
That means a quarter of the time, i.e., just as much as we put in our prior, we're staying put.
About 20% of the time, the agent wants to go and explore either arm because, I mean, there's still reward there.
So we're going to go to position 4.
And the most likely option, with a grand 35% probability, is that we should go and investigate the queue.
So go to position 4.
If we then compare what that looks like at t equals 2, that is, what does the agent believe it should do after the first time step? So if it goes and visits the queue, what should it do next?
Here we see that it is quite unlikely to stay put, sorry, to go to the starting position.
There's some probability there, it's coming from the prior.
It is indifferent between going to either 2 or 3, because even though we know that we're going to get information at the queue, we still don't know whether it is going to be right or left.
And that's represented here by these two being equally valuable.
However combined, it means we have about a 60% chance of going to an arm, which is probably going to be the right one. And then with about, again, a quarter, so about what we put into the prior, probability, we're going to go and look at the queue again.
And this is the same result. Yeah?
Yeah. So as you pointed out, one of the big contributions here is this direct policy inference. So let's just say that we have the probabilities of different state occupancies at time one and time two.
Yeah.
But how do we really retain the fullness of a path? For example, time one and time two might not be jointly independent. So for example, yeah, the probability of being at the queue in the, at time one and time two might not be just 0.35 times 0.26.
So what, well, this is, I think we can take a look at this.
See, there's actually a washer.
Uh, what's like, just, just to clarify the question while you're typing it. Um, if we're not explicitly tracing the branching paths of specific sequences of states and rather we're getting over each time step, then how do we know that just because something is 10% likely on here and 10% likely there, but then that doesn't mean it's one in a hundred together that those both happen. It might be more or less depending on how the paths actually flow.
Right. So the thing is here, we're inferring a distribution over possible futures.
Um, so we don't have any observations available. We don't actually have a full action perception loop. We're just doing planning. Um, and since we don't have any observations available, the best we can do is get a distribution over all the different possible futures that we have. Now we

could do that, but forget that, but like an exhaustive search. So yeah, I want this and I decide this probability or we could just infer it. So what this tells us, uh, is not, is like, it's not a sophisticated scheme. It doesn't have sort of this kind of factual thing going on. What it tells us is that if we want to optimize, um, this combination of, uh, Bethy and generalized free energy, expected free energy, uh, before we get any observations, what is the optimal distribution over paths that we could take? Okay. So the T2 distribution is basically T2 distribution as viewed from T0.
Yes. And then when we get to T1, we'll update, but we're basically taking, uh, something like a mean field or an aggregate over all possible paths to T2. And those may radically shift. Like, obviously, once we go to the queue and find out some information, then we're going to update from point 3.3. Okay. Yeah. So this is, this is at T equals zero. This is the agent saying, well, if I have to make a decision right now for the whole game, for all my time steps, for my whole plan, what is sort of the, the most likely thing that I should do and how should that distribute itself?
Once we actually start to get observations in there, um, then, then yeah, uh, these are obviously going to shift. Yeah. Then it's very notable that the two arms do have this high quantity taken together, but absolutely equal it. And that's that kind of like being precise about uncertainty that we would want such a planner to have. Yeah, exactly. What this is saying is the agent is really confident that it needs to go to one of the arms, but because it hasn't got an observation because it hasn't actually seen the queue yet. It doesn't know which one. So it assigns them equal probability, but it is pretty damn sure that it needs to go to one of them.
Yeah. Which again makes sense if we think of this as doing planning at T equals zero.
Yeah. It's, it's really cool. Like, especially after so much expected free energy, branching time, active inference, exhaust searches, just to see this kind of non path based direct policy inference. That's the contribution of the paper.
Yeah. I mean, I would say this is closer to a path based formulation. What you don't get is a search tree.
Thank you. Thank you. Yes. Consistent with a path based formulation of Bayesian mechanics, but not necessarily resulting in an exhaustive search tree.
Exactly. So what, what we can actually do here is we can look at the schedule.
What's going on here. So this, this actually shows us the exact way that messages are flowing under the hood in the code.
And the main thing that's different to most other approaches is that we have things flowing backwards.
We have these red guys here and we have these here flowing in the opposite direction.
So this information flowing from what we think the future is going to be towards what the present is.
And what this is telling us is given that I want to end up here and I reach backwards, what should I do?
And that is why we don't need a search tree, because instead of having to run forwards, we can do a forward sweep here.
So this is my distribution over things that I'm going to do if I don't know anything.
And in the other direction we have, well, this is distribution over what you should do, where you expect to end up.

And if you have where you expect to be and where you expect to end up, figuring out what to do is like what happens here. And that's how you do the policy inference.

It's because you have an idea of where you're going to be and you have an idea of where you want to end up.

If you know those, then inferring what you should do is, at least in these simple cases, quite straightforward.

And again, one of the contributions here is that because we do this backwards smoothing sweep, we don't have to do a forward search tree.

Because around these nodes, all we have to do is care about a message flowing in here, we get a message flowing in here, and then we can do our inference.

It also means that every time we add a time step, if I were to take this guy here and extend it once more to the right, I wouldn't have to, like an exponential explosion, I would have to evaluate the messages flowing around inside one of these slices once more.

So we need another backwards message, another backwards message, another forward message.

And that's sort of where one of the things I want to do, that we didn't get around to doing for these papers, is to investigate how this behaves when we scale to longer horizons and larger models.

Because this should scale much nicer than having to do an exhaustive search tree, because every time we add another slice, we just add the same number of computations again.

So let's go back here.

So what we do here in the last experiment is that we add these point mass constraints to the switch variables.

So what does this mean? So on a graph, it looks like this.

This is the exact same graph, but now we add these little constraints here, these little delta form constraints in the beads.

And this means that when we compute our posterior here, it has to be a delta.

So we need to pick one of the options. We can't say, I'm indifferent between going right or left, you have to pick one.

Because you have to be a delta spike.

So in practice, this ends up being the same mistake in the argmax of figure distribution over actions.

So you're effectively, as part of your inference procedure, picking the best policy and doing inference only with the most likely thing.

So let's run that.

And again, this gives us a much sort of clearer picture where we are now 100% certain that we're going to go to the queue at t equals one before we observe anything.

And then in this case, it picks two. But if you change the random seed a little bit, it picks three.

Because it's indifferent between these two guys. It just has to pick one.

Because we're enforcing that.

Yeah, it's still at t0 speculating it.

And so it's kind of like a bifurcation where depending on its random seed at t0,

it might symmetry break to imagine it going left or imagine going right in that implementation.
But either way, after t1, it's going to have the empirical decision.
So I might actually be able to show this as well.
If we do this.
So part two actually has an example of the kind of protocol where we have these.
Yeah, this is the thing I want to show.
We have these updates going on.
This one reminded me of the fire escape plan of a hotel.
I didn't think of that, but I can actually see it.
So, basically, what's different between these experiments and the ones we just looked at is
that these use here are clamped. So the little black squares and p.
That means we are picking a transition matrix. So we are fixing whatever our transition should be.
And this here, t equals one, is before the agent observes anything.
So this is effectively the same situation we just looked at,
except now we are evaluating a policy. We're doing the same search tree kind of thing
instead of doing direct policy inference.
So again, these are just transitions and these up here are fixed.
So we're no longer doing direct policy inference. We're just computing the free energy,
including GV terms, for this model without any observations.
And the thing I want to sort of point out is these composite nodes down here.
Is that the exact same that we just had?
Is that the exact same thing that we just had?
But you'll see there's nothing going on here.
Like these x variables have nothing on them.
But once we get an observation, it's the same as adding this constraint down here.
So we advance the time step.
It goes from one to two.
And then we clamp our observation.
And then we do the same protocol again, because the model hasn't changed.
What's changed is that we constrain one of the variables equal to our observation.
And then you keep on going to equals two and so on and so forth.
Every time you get a new observation, you just add these little constraints to the model.
And then you'll effectively get...
Yeah.
This reminds me of scientific pre-registration.
Like we have the whole experimental plan laid out.
We know what kind of information is going to flow in and we have a distribution or prior over the data.
And then when the data do come in, we just clamp the format that we expected.

And we don't need to do any structural changing to the factor graph.
And so it's like as we're moving forward and collecting data, we're just clamping timelines on the past and then leaving open what hasn't happened slash what we don't know.
Yeah, that's exactly what's happening.
Beautifully put.
Because that's exactly what's happening.
As we get data, we're just saying, well, now I know that this variable in my graph that corresponds to my observation.
I didn't know what it was before, but now I know that it was this value.
So I need to put that observation into my inference problem somehow.
My model is still the same.
It doesn't change the structure of how I think the world works.
Like some parameters might have shifted and stuff.
But the main thing that's changed is that I now have a data point that I need to add in.
And you can do that by adding these little data constraints, we call them.
So this here actually shows the exact type of thing that you were just talking about.
How does the model change once we get data points?
So this case here would be pure sort of speculation, pure planning ahead of time.
And this here is, well, once you start observing stuff, how do you actually add the information into your inference problem?

One other angle on this again, because I think this is really important.
The structure here is kind of like the timeless action perception loop.
And then we can be anywhere from at the top.
We have purely speculative, purely anticipatory.
On the bottom, we have purely retrospective.
All the data are already in hand.
And it's kind of like a solved issue, like looking at something that already played out.
Or you could be kind of the in-media res in the middle, which is basically just to say that part of it is already clamped in with the past.
And then part of it is speculative.
And all of those settings from like the purely anticipatory to the purely retrospective to in the middle of things with a little bit of both, all those are structurally described by the same.
Formalism.
Yeah.
And again, that's because the model doesn't change.
What changes is just we need to add in the data.
And what is kind of nice about this is that it also allows you to talk about doing different things at different

times.

And so one thing that we, for instance, looked at is once we have all the data, should we do our parameter updates then?

Because you can imagine a situation where you run an episode of something, like you are trying to do some psychophysics experiment.

And you have your participant run an entire episode of an experiment.

And then you expect them to go and update their beliefs about, say, some parameters for what occurs where.

You can model that by saying, once all my observations are clamped,

then I send messages towards my parameters, my Thetas down here.

So this sort of highlights the kind of things that you can start to be very, very explicit about.

When do you do certain updates?

What should the graph and the constraints look like?

All these things that can often have a tendency to sort of get swept under the rock a little bit.

That you can now be like very, very explicit about.

Yeah, that's super important.

Again, keeping with this theme of kind of like looking forward, looking back, being able to plan out the message passing logistics and scheduling means that you can have a sense of the computational requirements and so on forward looking.

And then also looking back, have a better sense of the way that variables were generated and trace back their kind of informational supply chain so that that's what makes it fully synthetic.

If it were fully synthetic chemical, we could trace from the precursors how it were generated.

This is fully synthetic active inference agent.

And so that means for every variable, we should be able to say, well, here's how we brought it onto the table.

And then here's exactly how the messages were passed from once it was placed.

Yeah, what is the model?

What is the inference problem?

What is the data?

And what are the constraints?

That's like, we should have everything be completely transparent.

Because that's, in my opinion, that's good.

The more, you know, we're allowing scrutiny of all the little different bits and pieces, and the more we can investigate how everything is going on, the better.

And this is kind of the ground floor.

Like, you could keep digging and ask, well, how is addition carried out?

How is multiplication carried out in Julia or however?

Yeah.

But this is kind of the layer that is the foundation layer for building then more elaborate graphs.

Like, we don't need to go finer scale or do we?

That's actually kind of funny, right?

Because each of these messages, we're drawing them just as arrows.

But each of these messages is a computation, right?

You mentioned that you need to do a multiplication.

Like, one thing we need to do for these models, we also do for this thing here, is we need to multiply a categorical variable by a transition matrix.

That is an inference.

That is an operation.

That is a message.

So there's a whole computation and a whole derivation that you can go and find for how to do that operation.

So everything is, should be, complete to transparent here.

Yeah.

All right.

I'll ask a question from the live chat.

Andrew writes, if the model is clamped...

Can I start the screen share?

Oh, no, it's fine.

It's fine.

I'll just ask it here.

Yeah.

If the model is clamped in this manner, how does the generative model evolve?

Is there a way that new variables can be introduced during the inference generative loop?

So that's a thing that's actually quite interesting.

And the short answer is not with this formalism.

There is some nice work on structure learning and how to do Bayesian model comparison, Bayesian model reduction and expansion, that you could use in this context.

But as is, what we've done is we've found a way to write down a model and an inference problem and the corresponding messages that allow you to do GFE-based optimization.

It doesn't tell you how to expand your model.

Now, again, if you want to combine this with something like Bayesian model reduction or expansion, that would be cool.

I think someone should really go and do that because that would be an interesting sort of avenue to go down when we can start to combine these things.

And one of the sort of things that I wanted out of these two papers is that I wanted to be able to get this kind of thing explicit.

I wanted to have a language that is not too difficult to pick up that allows people to ask these kinds of questions.

Because there's a very different, at least in my mind, I don't know whether you agree, Andrew, but to me, there's sort of a different vibe to philosophically thinking about what does the model evolve like and saying, well, how do I add a node?

One is a very concrete problem that you can start to talk about while the other can end up being very, very abstract if you let it.

I like things that are concrete and transparent.

So I'm really glad that this kind of question pops up because that's one of the things that I wanted to do with these papers is to get people thinking along these lines.

Awesome.

I'll kind of give another angle on that question.

So there's a implicit or unprincipled or ad hoc strategy here, which would be we're going to be operating our model and then we're going to have some extracurricular diagnostic.

Like if it's taking too long, we're going to then we're going to do something else outside of the generative model and change the generative model.

Oh, maybe we should add another variable.

So that's kind of like the non-principled approach of which there's an infinite amount of these types and they may or may not be documented in the paper.

Whereas the principled approach would be to bring that structural learning, that learning structure explicitly in through Bayesian structuring.

So then you could say, OK, well, there's either one, two or three factors in this regression and then we're going to have a distribution, a prior over whether there's one, two or three factors.

And then we're going to clamp eventually and find the best fitting model, whether it is one, two or three with the appropriate penalization.

And then someone says, well, what if there's four?

It's like, great, now we're going to explicitly bring that possibility into the model so that we have a unified approach to talking about the kind of sensory motor, lower level parameters and arbitrary nestings of structure learning.

Those shouldn't be outside the perimeter of the fully synthetic agents.

Otherwise, to the extent that it's outside the perimeter, the engineering perimeter, it's out of control.

Yeah, I think we should be a little bit wary about what we attribute to the agent and what we attribute to the engineer here.

Because when we set this approach up, an agent is just a graph that receives some observations and is allowed to emit some actions.

So there is still an engineering aspect here about how do you construct the graph?

What should the graph look like?

What should the model look like?

Which is not addressed in these papers.

But you're absolutely right.

There should be a principled way of doing that, which something like Bayesian model comparison is a way to do.

But the agent, the graph that does inference, just does inference on the graph.

At least with the tools that are in here, you would need to add something more to do, like an organically growing graph or something.

Does the graph ever undergo structural evolution?

Or does the structure of the graph change, but then through clamping and parameter updating, the graph can, even with a fixed topology, represent structural differences?

Or are there ever actual structural changes in the factor graph?

So I think that's one of the nice things about the CFG approach to writing down models is that you can write, like you need two things to do to minimize free energy function.

You need a P and a Q.

That's what you need to write.

That's what you need to write.

That's what you need to write.

And writing P is done with a standard factor graph notation.

So you write out your boxes and your edges until you construct your graph, your model.

And the constraints are then done in Q.

That is, you write them separately.

So you can talk about what happens in terms of constraints and what happens in terms of the structure of the graph, sort of separately and explicitly.

So the graph, at least in what we've done here, doesn't sort of intrinsically undergo any topological structural changes.

It's just the model you write down that you, as an engineer, as a scientist, is interested in.

What you can change is you can change the constraints.

That allows you to do different things with the same model.

For instance, incorporate data points as your agent sort of progresses through time.

What you can do then with this type of notation is you can be explicit.

You can be explicit about when you're doing something with the constraints of the model, on the model.

Sorry, constraints on the inference problem, I should say, where you're defining your Q.

And when you're doing something structurally with the generative model, when you're defining your P,

because one is done in terms of this bead overlay notation, and the other one is done with standard sort of factor graph methodologies.

So in that context, what's the main advantage of these two papers is that you can be explicit about what you're doing.

Yes.

Very interesting.

I hope so.

Continue with the code here?

Yeah.

Yeah, I mean, we've actually reached the end of the notebook.

This is all that is to the policy inference experiments.

But one of the things that you can do with this type of thing, if we go back to the generative model specification,

is that this here is just defined in terms of nodes and edges.

Now, you have my variables and I have my nodes that define them, and I have my goal observation with the P substitutions and everything.

But these units, they're sort of atomic in nature.

All a node cares about is its own marker blanket.

So these things are composable, meaning that if you want to build them with, say, hierarchies or heterarchies, you want to add multiple modalities, all these kinds of things you might want to do.

And that still works.

So with the caveat that there might be some coding bugs and stuff you run into because it's still software after all.

But theoretically, once you have these different things as little building blocks, you can sort of Lego your way through to any model.

It's something that we hint at in the papers as well that we're not really sort of explored in the experiments,

which is that this type of active inference works for freeform generative models.

That means because it's local, all the little changes we do are local on the graph.

Everything else can just be whatever it wants to be.

So you're no longer limited to using a POMDP model, which is an awesome model.

There's loads of things you can do with it.

It's a great model.

But there are things that might be easier to do, like operate with continuous variables, for instance.

That might be easier to do with a different type of model.

And if you can sort of Lego block your way to a generative model that represents how you think,

your agent should think that the world operates, that is something you can now do.

Yeah.

And then you can do that too.

Because the model sort of space has become much, much larger.

Right there with the how you think an agent should think is the articulation between the engineer and the agent.

That's the second level inference problem.

Yeah.

The first level inference problem is what the synthetic artifact will be doing in its runtime.

And then the engineering question is how to create that.

And so it's almost like the engineer is only interfacing with the inference challenge or environment through the blanket of the fully synthetic agent.

Yeah.

And that's, in my opinion, that's how it should be.

I guess if I put on my engineer's hat for a bit, I'm a lazy engineer.

I can go to, if I have a problem to solve, I could go to the literature and I could find, you know, 57 solutions.

And I could pick one and I could implement it, tweak it a bit, and I have 58 solutions.

That's one way I can do it.
I have several solutions for every problem.
That becomes unwieldy real quick because every problem you have like a boatload of solutions.
What I like to do is say, I want only one tool.
I can only do inference.
I believe that everything can be solved by inference if you set up the problem right.
And the trick then is how do you set up the problem right?
So as an engineer, what I would sort of take from this way of approaching problems is that my task is to build a generative model that does the thing that I want it to do.
And there's going to be some iterations and some trial and error and some tweaking of things and all these nice sort of practical challenges that we all run into when we try to actually build things.
But the tool should always be the same.
If I want to control a robot or track an object or do localization or preference learning or filter an audio signal or whatever you might want to do, the tool should always be the same.
All I should need to do is do inference.
So my job as the engineer to figure out what is the model structure that produces the results that I want.
Because it's clear if I want to, say, move a robot arm and I have a generative model that can only move to the right. I'm going to run into issues.
Now, again, as you sort of also hint that there's an extra level that you can go to where it becomes about structure learning.
Now, if you want to learn the structure of the model, you want to learn P, you want to sort of build your graph organically, which would be amazing to solve. But I'm not going to claim that we're anywhere near there with these papers. These papers is just a way to accurately write down what I, as the engineer, want to happen.
Is there any more code you want to show?
We can take a look at the other ones.
But they basically work the same way.
So this is for generalized free energy with an action perception loop, i.e.
And then a fixed policy. This simulates the kind of things you would get in something like Pymdp or SPM.
Excuse me.
So again, the setup here is very, very similar. Define our model, which has a policy that we want to evaluate. It has two time steps. It has our goal priors. It has our initial state prior. But now we're also interested in learning the parameters of our observation matrix. So we put a prior, a Dirichlet prior, over our A matrix, over our likelihood matrix.
And otherwise, these here move, like they look very much the same. It's instead of mixture models over transitions, it's just fixed transitions.
And this here is calling the exact same sort of P-substituted mean field constraint, all that

goodness node that we used for the last, like the direct policy inference simulations.
And then we do the same thing.
We load up our priors, we load up our matrices.
And because we want to repeat this over and over and over and over, we're actually going to run an agent for 100 episodes. So it's going to play the game 100 times and try to learn the likelihood matrix.
And does so by inferring an action.
As we're doing infinite or missing passing the model, taking an action, executing it in the environment, here and here, and then getting back an observation. And this getting an observation does this clamping operation that we talked about. So this is literally the thing that's shown here. T equals zero,
T equals, as T equals one, T equals two, and in this case, T equals three, because that's how many steps it's in the teammates environment.
Sorry. And if we do that and we run this 100 times, oh yeah. So every time the agent has completed an episode, so it's done, it's two steps. It gets to send messages towards the parameters. It gets to learn about the A matrix, the likelihood of the observations given the states.
And if we do that, we can look at the differences from the sort of very flat priors that it gets initially versus the n posteriors to look at what has this agent learned.
And the way to sort of read this plot is that O refers to the origin location. So location one in the,
in this thing here.
Yeah. Location one in this thing here.
So origin. We have the left arm and the right arm, L and R. And we have C, the Q location.
R, W means reward. You get a reward.
N, R means no reward. And this here says, is the reward to the left, the reward to the right.
Q points left, Q points right.
What we can see is that nothing really happens in the origin state.
We're always moving, so we don't really get much.
We can see if we're on the left arm
and the Q points to the left, the reward is also on the left arm.
We are pretty likely to observe a reward.
If the reward is on the left, we're likely to observe a reward. We go to the left.
Same thing over here. The reward is on the right, pretty likely to observe a reward on the right.
And the interesting thing up here,
so what happens at the Q location, right?
Because this says that if the Q points to the left,
the reward is also pretty likely to be on the left.
If the Q points to the right, the reward is also pretty likely to be on the right.
So this shows us that if we do this iteration, the agent can actually learn
the pretty good, we're going to say the correct, but like a really good
likelihood matrix that predicts where things are and how all the relations

actually work in the TMS experiment.

Which is nice, because that means that we don't have to put all this in.

We can actually do parameter learning using this GV-based message parsing as well,

which is really neat.

And here we have some free energy curves.

So this shows the free energy at the initial time step.

So before any observations have been received.

All right.

Yeah.

And when the first observation has been received,

and then after the second observation has been received,

then we do parameter learning.

You'll see everything is nice and decreasing, but there are these spikes here.

And these spikes indicate when the agent predicts a win, but gets a loss.

So the agent, the Q says the reward is to the left.

The agent goes to the right, so to the left, but it doesn't get a reward.

There's a bit of stochasticity in the teammate's environment.

So it's not always 100% sure, even if the Q points the right way and everything,

that the agent actually gets a reward observation in the end.

So we see this is known to make this

do the p substitution we don't introduce the gf term we just run box standard

a variational betty free energy minimization

and the model looks the same we just use like some betty options for uh this composite node

what is meta so meta is a way that's all done in rx infer

meta is a way to pass options to a node and in this case the betty meta tells

this sort of joint node composed of a likelihood model and a goal prior

whether it should optimize generalized free energy or betty free energy

so you can think of it as if we look at the graph again

and if you have this node here we have to make this a little bigger

we have this sort of composite node here inside the dashed lines
there's a difference between if we draw a circle here or we draw a square
a square would indicate that we do a p substitution which turns this locally into a generalized free energy
if we don't do that draw a circle don't do the p substitution what we're left with is a mean field factorized betty free energy
and the way we do
this sort of switch between a circle and a square is through this meta object
uh you also use it for other things for other nodes in rx inferred like if you have
say a non-linear function that you want to use somewhere in your graph you want to pass messages
through it you're going to do some approximations most of the time so you can say pass your approximation
method uh through the meta object and if you know run some iterations or into sampling or something
that kind of information is something you can feed in through the meta object
so what this thing is really saying is just swap uh like the square for a circle
so that we do betty free energy instead of generalized free energy
and then everything else here is exactly the same
this is literally a copy paste of the the other notebook
so that's what we're doing is we're doing is messing with the constraints
the model is the same all we're doing is messing with the constraints and the sort of p substitution
and if we instead of saving the figures actually create these
the values yeah you'll see this agent only gets uh stuff
like around the the outcome because it starts with uninformative uh priors over the a matrix
doesn't really know what the reward is there's no incentive to go and explore
it just doesn't really accomplish much you can see the same thing free energy is
it's flatlining nothing is happening and it is consistently not performing well it is
it just endured a hundred trials of loss i don't think it is particularly enjoying itself
and again so the the thing that we can illustrate with this um is the power of messing with uh
constraints and messing with p substitutions and messing with these sort of local adaptations to the free energy
energy because the model in both cases with the p the graph is actually the same there's no difference
the thing that's different here is we're messing with the constraints and we are messing with uh
turning this local beth free energy into a local generalized free energy
and that produces these quite drastic changes in the behavior of the agent
yes the beth free energy agent does not do well here just changing that one
aspect moves the epistemic
phenomena into the present that's kind of like efe like component of the gfe that we discussed last time
yeah exactly so what this uh thing does uh basically if you write if you do the battery free energy
uh you end up with an agent that does kl control
this ends up being a completely standard uh control problem like controls in for instance there's

nothing nothing fancy going on here um and crucially that does not include an epistemic component we get no information seeking and we see that here
there's no information seeking anywhere here there's no information seeking anywhere here
there's little guys just flatlining
but once we start to mess with the function that we introduce this epistemic component
we get expiration we get good performance we get this active curiosity information seeking all these nice goodies that we know and love about active inference agents
and we can show that very very directly by simply saying well i'm going to change this little circle to a square
and then we have uh some aggregate experiments that we also ran which is just repeating the same thing over like numbers of agents because there's some stochasticity uh sometimes they get rewards sometimes they don't if you want to compare um yeah how this does the aggregate you need to do multiple runs
and you can see the same thing that you need to do multiple runs you need to do multiple runs
you can see the the random earlier stay and it took like an hour and 20 minutes
um but what we can do is we can look at the distribution over
when say how many times per run does an agent obtain a reward
and you can see that it is very much sort of skewed
and then we can see that it is very good to the right which is good that means the agents learn
they learn the parameters and after they learn the parameters they consistently get rewarded on their
uh on their trials it doesn't always happen right there's a bit of sort of tail end down here
and what this tells us is that sometimes these agents um because there's no consistency in the rewards
they're just going to get stuck in some unfortunate minimum they get some conflating information they never quite manage to recover and that that can happen sometimes you learn something that's wrong
and you sort of end up sticking to it
so it's not a bulletproof thing
uh the same thing here we can show this is sort of the the ideal uh expected reward you can get that as the trials progress on average agent starts learning earning more and more and more and more reward
how you get closer and closer to this line there's some wiggling in here that's because we have to use a sampling approximation when we send the message towards the parameters so there's some stochasticity in here which is why we get this sort of bit of wiggling that and the rewards so this basically
shows that as an aggregate um agents get better they get consistently better
uh
but again the model is all the same nothing changes
cool good with the code yeah i don't i don't have any more code uh not for the for these papers at least cool yes thank you then so i'll stop the screen share
okay i'll um share screen um i guess just to sort of transition into this second component like when you sit down at the table where do you begin
um usually i begin where i left off the day before
uh

that's worked for uh for quite a while now
and so there are other things that i'm working on right now
uh that i i can't really talk about yet
but just as a general a general um building your own generative model like if you had a new fun
non-non-proprietary study where would you begin with sketching out what to be on the road to to
working with it this way right so if oh now again yeah so the first thing i would do
is that i would start drawing graphs i will start drawing my my constrained ffgs and see if i can figure
out what is the problem that i want to solve and then get that real accurate and that is not a trivial
exercise um because once you have your model specification and your inference specification in
place then like the the heart to me the heart part is done
because again if i put on my engineer's hat the main thing that i have to do if i want to build an
application i want to solve something is that i have to come up with the graph and i have to
come up with the constraints that define my inference uh problem once i'm there i need to figure out are
there any parts of this problem that i can't solve is there a message that i need to work out is there
some approximation that i need to figure out to put in there and if so it's nice that's sort of a
uh like a research question on its own quite often um but if there isn't then the next thing i would do
is i would try and run inference in my model actually does this model actually behave as i think it does
yes especially if you're in the game of designing active inference agents
uh sometimes these things have a mind of their own uh several reasons why that could be
um it may be your intuitions about like the thing that you wrote down uh don't actually quite
hold up to to how things manifest in the real world or there might be like bugs in your code all these sort
of very practical things but once we're at this place um like to me a lot of the heavy lifting is
done because the hard part if you want to build it just me if i want to build something specifying the
model specifying the constraints and figuring out to solve the inference problem once i have that then
everything else is sort of implementation details
um it's a funny story we had this during the design of the experiments uh that we just looked at
so if you look at the transition matrices that we use uh if you have a the original teammates
transition matrices they let the agent get stuck in positions two and three
the sort of attracting states once you go there you can't go anywhere else and the way that is done
practically is that the transition matrix sends you from state two to state two whatever transition
matrix you choose you choose if you do that with our setup that means you allow to send backwards messages
uh let's say well if i pick action one which is no go to state one and i'm coming from state two i would
also go to state two does that make sense so then what's the issue or what did that result in
um the result is that the agent starts inferring some very very nonsensical policies
uh because it'll say well i know the goal is at position three but i'm going to confidently go like
pick the matrix that corresponds to go to one or go to four go to two and that's because all these things
all these matrices allow the reverse mapping from say uh state two where you think the reward is to any of

the other states
why is there no reverse constraint
so i kind of put it because if you're reasoning backwards you have to think about how do your
model run in reverse and what does your transition mean mean the other direction
and the way we solved this was that we sent uh we let every sort of invalid transition send it back to the beginning
so every um yeah basically if you end up in one of the the rewarding arms
uh every transition that you pick is going to send you back to the stars it's just going to reset the
whole episode and if you do that you get rid of this problem of the backwards message not corresponding
to your intuitions as an engineer because we're designing this thing to say well one matrix is
go to one when we just go to two and so on so forth but the math doesn't really care about our
intuitions the math is like does what the math does just it's up to us to make sure that we write things
down accurately okay just to kind of confirm this so the initial strategy was you used as kind of
classical in pomdps you just had the state of reward be absorbing however it was almost like you could get
in but then you didn't want it to leave with the absorbing state but you actually you had like a
ladder out with the backwards message and so that was leading to some weird things so instead you change
it so that the reward state brought you back to the beginning essentially tucking in the end of the
trial to something like the beginning of the next trial and then that better cohered with intuitions
around how policy should be selected so you can think of it also as the backwards message saying
um if i go here where should i come from
so if you are in say the rewarding arm where should you be coming from
and if you allow the rewarding arm to map from itself to itself because you make it absorbing
the agent can correctly because that's what you put in infer that it should be coming from the
the rewarding arm when it is in the rewarding arm
yeah how do you you can see how that's how do you get a million dollars step one start with a
million dollars exactly and that's not the kind of inference that we we want our agents to make
it's right if you have a million dollars the most likely way you got them is that you had
them a second before you had more a second before
at least a million dollars a second before but yeah that that kind of inference is something that
we want our agents to to not do and that's an issue that doesn't at least for now sort of research
hadn't arised did not arise when we only did this sort of forward rollout version because if you
never reason backwards this kind of thing just doesn't pop up but if it starts then you need to
consider this kind of sort of circular uh step one start with a million dollars type of reasoning
this is an example of something that we ran into in the course of this experiment where we were pretty
sure that you know everything was sort of set up correct and it was but the agent didn't do
like did not behave in the way we anticipated and sort of how we wanted to behave so we had to go and
look at well what what is actually happening not what we think is happening but what is actually happening
when you sort of start to look at the code let me ask a related question from the chat andrew says

what you are saying is there's no conditional probability placed on where the agent thinks it is
when doing that reverse message uh is when
i think is the question
is there a probability placed on the agent when it is doing the reverse message
yes but it is conditioned on the future
you're doing everything backwards so it is condition sort of conditional on where you should
be in the future
you can think of sort of the the forward rollout approach as saying well if i am here and i do this
then i go here and so on so forth then you have this large tree that sort of expands to
where you could go and then you look at all the places you could go and say well that one that's the
one i like you can also think of doing the same i'm sort of i'm approximating a little bit because we
don't actually do search trees but you also think of doing the same thing in reverse right you can say well
if i am if i have already obtained everything that i want in my life
where is it most likely that i came from
and i say all right if i go one step back
where is it then most likely that i came from and so on and so forth
and that distribution that kind of logic uh is what we're doing with the backwards paths
so then when you have the forwards path saying well this is where i'm likely to end up
the backwards path saying well this is where i want to go
you're solving that difference is is what you do when you infer the action you know two messages
colliding and they spit out something towards your um your action variable
yeah okay what that's making me think about is we classically think about new data coming in we're
conditioning on the existing conditions and then new observation comes in things update so that's the
sort of march forward then there's this kind of aspirational or fulfillment based conditioning
conditioning on not what has been sedimented and brought forward but conditioning on what is
anticipated or preferred and then yeah let's think about the the um three time step case
where there's a clear last step and there's a clear first step and they just meet in the middle
and it just like the sort of um the conditioning from both ends results in just this obvious solution
however also it could be more ambiguous at the beginning of the end or it could be more than three
time steps and then that's where we get this um message passing and kind of annealing as data are getting
clamped through time and that has a relationship with the search tree based methods in that like the
territory of what they're explaining is the same sequences of action but again to be really clear this is not
doing policy specific evaluations followed by comparison of expected free energies or any kind
of enumerative search at all it's solving one direct policy inference problem as this kind of annealing
process so i think the beauty of this approach is that you can do both
both i like to do direct policy inference because i think it's it's to me it's it feels like the right
way to go but one of the things we do in part one and to extend demonstrate the experiments in part two
is that you can recover like these forward rollouts uh these classical algorithms

as special cases by only passing messages forwards because when you have these p substituted uh goal observation nodes with your mean field constraints and everything the energy terms of those nodes ends up being equal to one that's what we talked about last time and so being equal to one term in the efe of matching policy so you can reproduce like the forward rollout method by saying i'm only going to pass messages in the forward direction i'm going to fix all my action variables and then i'm going to compare all the energy terms of these little gfe nodes and that will reproduce you know the exact algorithm that we've all been using for uh for years you can also do is you can add a backwards pass what you can also do is you can say well now i want to infer my policy and so on and so forth but because this is a toolbox um it's just a thing it's just a way of thinking that allows you to do different things um so the policy inference is not inherent it's something that you can do uh if if you want because it's just an inference problem if you don't want to and you have good reasons for doing something else um you can go and do something else

um preserve to to rub back a little bit to me this idea of reasoning backwards also works with sort of my intuitions on at least how i as a person often operate

like i'll have some idea in mind i have some goal that i want to achieve

which could be you know i want to get a phd or i want to make a taste of dinner or something and then i reason backwards well i need to do this i need to do this i need to go and buy all the things that i need i need to set aside time for cooking and so on so forth

there's sort of a backwards reasoning that if i want to go here i need to do this this this this this and then there's the forward planning all right if i am walking out the door i need to go to there and then there and if i keep walking i'll end up in the right place and so there's both components to it if you want to put sort of a more anthropomorphic uh intuitive spin on it

in practice uh what this ends up being is just inference on a graph and if you're an engineer it's up to you to design uh you know your model and your inference procedure

in a way that uh you end up with the results that you're looking for

yeah this about the policy selection here the forward pass policy is basically habit coming forward because in the absence of any aspiration or preference

policy is just well there's various ways you can select policy still you could select maximum information gain you could select different things but the prior on policy the e vector is habit so it's kind of like that's the starting position for what actions are selected from now

and then it's just really interesting to think about this

forward and backward solving which is of course a classic technique

but again being done in this way where you could recover tree search but it isn't tree search

so the the the thing you would recover is a trajectory so an evaluation given a particular policy

the tree search uh comes about when you want to try and expand this and be smart about uh how you compute things for different given different policies because obviously you can do that for every single policy one at a time and evaluate everything but that's even more inefficient um so what you do practically is every time you sort of reach a branching point you need to select a new policy you say well

i'm just going to take my prediction here of where i would be and i'm going to branch off into all the different um policies that i could pursue and that'll give me sort of a new set of branches on my tree and then you do the same thing so that's how the search tree comes about but each sort of path through that search tree would correspond to a forward rollout on one of these models
yeah
you said inference is something that you can do it i want to go a little bit back um because you mentioned this thing about the e vector um which i actually think is really really interesting i don't know since you have the screen share i don't know if you have the the papers available which papers oh of these your two yeah yeah do you have a part one yep all right can you go to page 33 of course thank you here this is the one i want so if you look here um we actually have an e vector
sitting right on top as a parameter for the indicator variable
and what we're saying here is just we're literally putting a prior
on uh which policy like which action to pick at each time step but that's that just comes about because we're building it out of these little graphical building blocks
so if if you wanted to you could put a prior on e and then you could learn e
that that's absolutely something you can do or you can say have
um a shared e with its own dynamics so now your habits are time dependent and that'll be another sort of thing that moves along at the top with its own state space the night was everything now these are the kind of things that you can start to do
um we used flat priors here but you could just as well say i'm gonna sort of intrinsically have my agent be really really biased to go for the goal whatever it takes
uh we could even say i it's gonna prefer the leftmost goal more than anything in the world
and that that is something you can do but what you can do here is you can be explicit about it exactly how your habits work exactly how your e works um practice you know the sort of semantics intuition you can attach to it is the same as they they've always been but you can start to look at well let's look on my graph what is the thing that i'm updating what does this like actually sort of specifically concretely mean and you can draw it and if you wanted to do something else you can you could for instance imagine that you have hyper priors for e set by another agent that's means you extend the graph and maybe that agent has his own observations and so on and so forth so then you need to attach that to the edge that is currently clamped with e
and now you're having an agent that sort of influences prior preferences over policies or habits if you want to say well now these need to update at different rates that's you having to mess with the schedule in which you pass messages sometimes you pass more and like one agent than you do in the other but these kind of things can be explicit now that's to me one of the the big sort of contributions that we wanted to put forward with this paper is you can now put all of this sort of directly on the page unambiguously what exactly is happening you notice for instance here we have different e vectors for e at t and e at t plus one that's also a choice
you could for instance have these be linked together and say they have to share the same thing

um and that same thing now has a prior that that's something you could do and that would be
a completely valid graph and you could figure out how to do inference in that
so like the idea of opening up freeform models means that um model structure it doesn't have to be
like a b c d e f g and so on and so forth we can do whatever model we want like the space has opened up
uh for uh yeah the things we can try out with with active inference
all right i got a little bit of a challenging question nice you're talking about active
inference and the toolbox and that sometimes is seemingly not what people expect they come to
active inference or the free energy principle looking for an account of a human experience or
of a certain cognitive system which of course isn't to be found in the toolbox you know any more than
the blueprint or the house itself is to be found in the toolbox so could you from your view going back
as deep into history as you like can you give an account of the active inference toolbox what tools
have been brought in when which tools were versioned how which tools were deprecated how like when would
you trace the active inference toolbox coming to existence with what just to get a little bit of a
run-up to the state of the toolbox today i think pretty much anyone you're gonna ask is gonna have a
different answer to that there are some sort of big fixed entities that we can all look at the no the
the biggest one of course being spm uh which has been the go-to tool for many many years and in many
ways still is because it is sort of the the battle-tested version with all the bells and whistles that
carl himself has large parts written like that would be sort of my my
my um proposal for a grandfather grandmother software package for how to do active inference this
should be the gold standard in my opinion
um later on we've had uh pyndp which was uh largely implemented by by connor heinz and i like chance and
a couple of other collaborators which is a great project uh i've used this it's awesome which is a
python implementation of many of the routines that exist in spm um so this sort of brings many of the
tools to python instead of matlab which is a lot more accessible a lot more popular these days
um and that was sort of to my mind be the two big active inference toolboxes in bias lab we have
rx infer which is a general purpose message passing toolbox
so it is not specifically an active inference toolbox you can use it to do inference and you can use it
to do a lot of things that we would like to do with for active inference but it is a a message passing
so the patient inference engine first before it is a sort of specialized active inference toolbox
which is great because then you can use it to build your active inference uh parts uh like we did with this
uh a constrained goal observation there that we used for the uh the simulations we just looked at
that's using mostly stuff that is an rx infer and then one little node that we uh added on that does this gv based
thing i think what but to my mind is um the more interesting question is the types of algorithms that have been employed
because again and i'm going to miss some here and for everyone that i don't mentioned uh i'm sorry
uh but like from the original sort of inception in 2015 of the first active interest routines with these sort of

forward search tree uh through to um something like branching time active inference or sophisticated inference

or there's been a lot of really really interesting work with monte Cala tree search uh

and the whole sort of deep active inference field that i haven't touched on at all in in any of these discussions because it's it's a again it's a different approach um there's sort of been an explosion of really really interesting algorithmic work uh on how to to do this that how to solve the the active inference problem in a smart and interesting way

and the things that we introduce here are sort of more in that tradition where we're saying instead of trying to find ways to be clever about the search tree uh or trying to sort of like the sophisticated inference for one is a really really interesting idea but it's yeah it's a different thing instead of being clear about the search tree we want to introduce this backwards path so that we can solve everything as part of the the inference problem

um like parts of this is also something you can find in uh there's actually sort of an interesting thing as well that's mentioned in the papers if you go and look at uh power and for instance 2019 paper where they introduce the generalized free energy

uh everything is still conditioned on a policy but they the message passing schedule the forwards backwards thing is actually present and the update rules are based on generalized free energy so they end up looking a lot like the update rules that we derive it's quite interesting you can sort of see uh the seeds of what we based this work on quite clearly like you can look at the equation side by side

um which to my mind is really really interesting because that was sort of the the inspiration like you have these update rules you have this forwards backwards thing what if we try and do this but we actually come at it from uh the point of view of doing message passing on graphs

um so i agree this is a big question and i'm not sure i can do it justice because there's so there's been so much interesting work on um software and algorithms and simulations for active inference that whatever uh parts i bring up i'm bound to have missed something um but if you if you put me on the spot these are all the things i would highlight i would highlight spm i would highlight pymdp and later on uh you know like the message passing toolbox boxes from uh from bias lab algorithmically i would uh highlight the the original algorithm along with all the things we have here like branching time onto carlos sophisticated uh deep sampling based all these kinds of things and then put our uh work in that field more as this is a different approach to solve the problem that we're interested in when we want to design active input agents that do uh do planning in interesting ways with all the epistemics and what we like

i don't know who asked that question um but you're right it's a tough one it's kind of a perennial question i mean even with some familiarity

sometimes i find myself grasping at how simple and clear the problem is and how we have all the tools to work towards it and then there's just this interesting moment like we know what we're interested in and what we want to do we also know how to do it so then like what do i do then

when we know what we care about and how we prefer to be and seemingly have no limitation on the actual composability and the implementation these developments happen year by year

yeah i mean that that's always going to be more i expect that our method is also going to be superseded by something that's even cleverer and even smarter and i look forward to to figuring out what it is whether we figure it out or whether someone else figures it out um whatever it's going to be it's going to be awesome but i also want to dwell on one of the the points you just mentioned with this idea that if you have what looks like it's such a very simple problem it doesn't mean the solution is simple because most of what we're doing like 90 of everything we care about here is really just solving base rule like base rule is pretty simple to write down um the trick is that it's really really really hard to solve that's why we have whole fields of variational inference trying to come up with approximate solutions that are good enough why we have uh all the like really brilliant interesting work on monte carlo samples and all these kinds of things it's all because we know the problem is the problem is easy to state problem is just it's all base solution can be really really difficult
and that's sort of where the the the devil in the details and all the work goes in
same thing i just want to just want to minimize free energy it's easy well in paper paper edits but if you actually have to solve it you need a lot more paper
easier said than done that was probably true about the first uh rocks being chipped into blades just make it narrower just make it sharper yeah it's easy it's easy just you know go go hunt the mammoth it's easy just made it make it lie down you can take all the food no but but i think this is sort of um to me one of the really really interesting parts about working in this field um that oftentimes it's really easy to state what you want at least for me i have i want to be really really good at updating beliefs and i want to do that in a way that's consistent with all these things that we like but really when it comes down to it we just want to update beliefs in a good way
how to do it and the details and how the belief updating should work and whether we should introduce all these extra little bits and pieces like information gain terms all this kind of stuff
to me that's where sort of you you get to dig down and immerse yourself and find all the cool things and try stuff out and sort of that's where the work is that that to me is really the exciting part but i like that i can always step back and say well i just care about doing inference man
let's look at where you left the papers off as we kind of think about where we go
um so other than broadly summarizing all of the acronyms and terms
that's a lot you
clamp the paper in a very interesting way at the end yeah so what what would you say about this final paragraph and like how did the different authors perspectives um arise to result here
so this is actually an interesting point um because i think i want to come at it in another way um what we're trying to do here is we're trying to provide tools we're trying to build really really good tools for doing active inference but as you mentioned earlier people come to active inference and the free energy principle for all sorts of different reasons and for all sorts of different backgrounds they maybe not everyone is interested in in building tools and the measure of whether our tools are useful are whether they end up being used by other people if you or others pick up this notation and pick up these types of message parsing pick up this grant and active interest

formulation then this this paper uh is going to have the impact that we would like it to
if nobody picks it up it's just a cool hammer that that keeps lying on the shelf
and sort of paradoxically um i think one of the barriers to uptake could be that a lot of the
people that come to active inference come from say neuroscience or more sort of philosophically bent
uh which is which is great like we need everybody on this train
but they might be concerned with other things than we are
and we didn't make any claims as to whether this is like a biologically plausible
update algorithm which may be something that you care a lot about if you're a neuroscientist
you don't have any claims about how this ties to experience or qualia or consciousness or all the
kinds of things that you might be more interested in if you come at it from sort of the more philosophical side
um so these tools uh is the best we could build as engineers trying to make good hammers
uh but if what you're looking for is not here if your um priorities lie somewhere else
then these might not be the right ones and like and that that's okay
um so yeah i think paradoxically one of the things that's uh could hinder uptake and the impact of these
papers is that the people for whom we're building tools might care about different things than uh what
provide with what we built i if you really really really want to use a screwdriver it doesn't matter
that we have the best hammer in the world you still need a screwdriver
so because this is a a again a tool building exercise because this is about doing things from
an engineering point of view uh that means they are tools for engineers so we don't sort of make any
claims as to biology or consciousness or all the other things that people are studying under the the
heading of active influence and the free energy principle we'll let people who are better qualified
and in many ways smarter at these things than we are take care of that what we want to do here is
just say here's a really really good hammer uh if that's what you want then we got you covered
these are really important i i i really respect that approach i think in terms of the active
inference ecosystem even if there seem to be like some barriers or hindrances associated with
really the articulation of the tool and the engine and the end what it's used for um this is how we bring
more perspectives in and if the hammer is oh yeah but then we also included this and here's this
proprietary phone home and it only does this type of nail and it only makes houses all of a sudden it's
it's not really a hammer anymore it's become welded to a larger apparatus um and yet if the head of the
hammer and the handle aren't connected it's not a hammer so then it's a fine line between like what
algorithmically and implementationally does this essential active inference
tool do what is it and then what can it be done with it and i think actually
rx infer and bias lab other than burr and your all's incredible work over the last years by situating
within a broader bayesian message passing framework that has helped first off bring in insights from
the broader bayesian inference space related to literally four knee graphs message passing reactive
message passing and so on and then also it helps us separate out the active inference models from other
models and there's nothing wrong with other models if you are building a model you find that your system of
interest doesn't require like an active epistemic agent for every single component

no one in the active ecosystem is going to get mad at you for like just doing what the situation demands and so that's very cool and again one of the the things besides uh the new messages and everything that that allows people to do this uh type of message passing based inference one of the things we wanted to stress is that we wanted this uh constrained ffg notation to be something that is quite easy to pick up like it's easy to write down your graph and if you want to do just bog standard belief propagation you don't really have to do anything like it'll just it looks like 99 like a normal factor graph but if you want to say do a factorization here and there you can like you can just draw that in by adding a little thing for each of the uh parts of your graph that factor together so we wanted to say we really stress this should be something that is intuitive and usable because we want this to be applicable for people who don't care about working out the intricacies of message parsing and factor graphs this should be a tool for everyone and if you come at it from a different background and what you want is a nice and clean modeling tool it should be that for you

what else do you think could come into play even just totally speculatively or arbitrarily like what do you imagine in the preferred active toolbox even if it's some sort of like synthetic intelligence fantasy like what would be the coolest thing that we could bring a new colleague to the table and set them down to to me i would like um to generalize all of these update rules that we have derived like the general form of the gfp based messages to a point where you don't have to worry about whether the thing you need is is there and then i would want it to be so that i can give it to you or i can give it to you know a neuroscience colleague and i can set them down and they can draw their graph in their cfg notation and all of that should get translated into executable code because then we end up in a situation where if what you want is a modeling language that solves your problem you can do that you don't have to be a technical expert and for those of us who want to work on the technical side of things there's a code base that we can open up and dig into but in terms of a toolbox that i would like to have have in existence it should be something that is really really easy to use generic um and fast because fast is cool do you see any difference or reason to build sketches in the factor graph form or in the base graph form we know there's a analytical relationship that links them but is there any reason to like work on building intuition about factor graphs specifically or would it suffice to build intuition for learners about base graphs knowing that they could be rendered to factor graphs later i actually had this we had a long conversation about this with a colleague of mine just just this week and i think if you dig deep enough the representation doesn't matter that much the update rules end up becoming the same i would say factor graphs uh like to me they're preferred i mean obviously i'm biased because that's what i work on but i also think they allow you to write down more if i have a base graph i can see that two variables are related because i have a node here and i have a node here and an edge between them the factor graph i can also see what the relation is because i literally have a little square that with a simple end of it tells me the relation and also if you adopt the cfd notation you don't just write down your generative model you also write down uh your constraints on q so you can write down your um like the exact inference problem that you're trying to solve and that is something that i i'm not aware of a way to do on on basin networks

so to me i think there's more information contained in a factor graph especially if you adopt constrained ffgs and then again as you say you can always have a one-to-one mapping between uh like a base net and an ffg so okay to to kind of replay this because there's different ways to constrain a given base graph there's actually like a few to one projection of factor graphs as constrained down to a same unconstrained base graph and so in that sense they can convey more information if you use the cffg and yeah so again they don't call it bias lab for nothing it's definitely your fellows's uh take but it's again this is also one of the things we introduce with these these papers um is that we identify a need to write down more than just the generative model if you want to be clear about everything else you're doing because if you don't have a graphical really you need to put it in equations you need to put in text and you need like half a page of hieroglyphics to convey what you want to do um and again that's a barrier if you want this to be an accessible tool for everyone which is one of the things that i want um so we augment ffg notation specifically in a way that i'm not aware of anyone that has done for for base base base nets you might be able to um so in that way you're right there's more information definitely in the cffg than in the base net but in general there's also more in a factor graph because you have to be explicit about the relation between variables more than you do with at least the versions of base nets that i'm familiar with

uh again i would say as i put on my practical hat you should use whatever is best for for your problem whatever you're most most comfortable with because a lot of the math you end up having to do comes out to be the same like in the end it's just it's just a picture yeah yes there's there's more to my mind there's more information in especially constrained ffgs uh which means you can be really really accurate and precise about what you want to to do by just writing down the picture

yeah a lot of thoughts there one great you left the door open to somebody introducing a graphical or other than graphical notation for base graph to create a true one-to-one map between base graph and the constrained for the graph it's not impossible it might even be trivial um and then it just makes me think of like a learning language that's more broadly used base graphs where the nodes and edges have a semantics that are equivalent to what is usually considered as structural modeling yeah and then you enter into the learning language and and play space and toolbox and then there's this strong relationship between the learning language again just kind of hyperbolizing this and the engineering implementation or specification and then from there it's actually anchored in message passing it's like that's how we connect the base graph as sketched to the message passing is like enter the problem into the base graph if you want to enter at that point transfer over to the cffg and then run the messages from there however if you wanted to you could just work directly from the fact graph and a base graph would have never had to enter into it i think i think that's doing a little bit of a disservice to to base graphs i i have my favorites um but if you go and look at like the original win and bishop papers for instance where the original divide drive the vmp algorithm it's all done for base graphs it's all done for patient networks um so like it's not that you can't do message passing again this is just different representations of a generative model and sometimes one is more useful i find that generically uh factor graphs are more useful uh when i want to work with message passing but some other

people might might disagree um what i can say is that uh the versions of base graphs that i i've seen i contain less information than a constrained ffg because you don't write down uh the exact inference problems i.e the constraints and the data points and everything so i i wouldn't say that uh you know there's no room for base graphs like base graphs are great uh but i'll say for the kind of work that um i'm interested in factor graphs feel feel more natural thanks for that that's great let me ask one more question in the um in the graphical brain 2017 yeah paper um there were three graph types mentioned so we've spent a ton of time talking about the the complementarities and the transform abilities of base graphs and 40 factor graphs could you say a little bit about this last form of graph the neural network or neural circuits um i am not intimately familiar with the formalism of neural circuit graphs uh uh so i don't think i can add anything above and beyond what's especially if i want to make sure that i don't say anything wrong fair enough fair enough okay just wanted to check so yeah that's fine they're all like different projections so but just wanted to ask i think sort of if you wanted to try and tie a bow on it um i would say the different graphs the different graph types and different types of graphical notations are useful for different things um and whatever you need to do you should pick what is best for the kind of problem you're trying to solve like if i have um if i all i'm interested in is dependencies between variables i just know i want to know which variables depends on which others base nets might be the best way to represent that and you know that that might be the case for the problem that that someone is working on that this is sort of the central information that they care about if i also want the relations between the variables to be explicit and i want to do message parsing um maybe an ffg or a cffg is a better representation and if i care about say neural implementations maybe like a neural circuit is the right choice again i don't want to sort of come out and and tell people what they should do because i think everybody knows what they should do better than i do what i want to sort of propose is that there is a a type of notation that has these nice properties you can easily represent the relationships between your variables and you can represent a constraint in q thereby also the resulting message parsing algorithm which is something that i find really useful and that i hope other people would but if that is not the type of problem that you're interested in and something else works better for you you should use something else
yeah again this is all pragmatics this is just a notation for how we we want to get our ideas across clearly and in the end what all this is really designed to do is to communicate information in a nice way and you know the the language we would choose to speak in that should reflect what we want to say we could try you and i and have a conversation in german but i don't think it would end well for either of us at least not for me i'd be there to hear but i wouldn't listen or do you speak german no nor do i i don't think it would end well for you to try there you go so again we could try and some ideas might be more expressible in german than in english but if if none of us speak it it's not going to aid us in transmitting information so for this setting english is probably the right choice of language for other settings it might be baseness and for other settings it might be cffgs i have a bias to watch cffgs because i think they are the nicest way to represent everything that we want to do but um for other people it might be different and that's okay yeah we just did a very long unpacking of the hammer

and again as discussed if you're not looking for that function it's not the right language or process
i mean it's just such an important higher order points
that that clarifies the contribution of the paper and the research direction and
offers up something to hopefully many people who will use it at least from my side only in spoken english
active inference ontology dialect i hope that people do use these tools there's only seemingly a few
individuals who are working in rx and for making active inference models it is very few relatively
so having that number broadly increase yes also training up different kinds of synthetic intelligences to do active
inference modeling that enriches and enlivens our ecosystem of shared intelligence
and the engineers also need to have hands-on experience and know and work through simpler examples to build
intuitions for larger systems yeah less larger systems be so far beyond our understanding that that
it's not useful or helpful yeah and so before we end up wrapping up um i want to reiterate the the
invitation that i gave last time as well if anyone out there is interested in working on these things
uh hit me up i'll be happy to help you work work after graphs
yeah because i think it's important that you know measure of a tool whether it's useful is whether it's
being used so if i can assist people in picking this up i'll be more than happy to
that's the most helpful and pragmatic
thing i believe i guess just in closing like
what loads up on pragmatic value for you what loads up on epistemic value for you in our field as we
head into like 2024. i think i'm in the very privileged position that i can do research for a living
so my pragmatic value and my epistemic value tend to overlap a lot if i can figure something new out
um that's pragmatic because that means i can write a new paper and that's the thing that i'm i'm producing
uh i think that would be my answer i want to try and see if i can make this practical uh
as i work it out for more models and try it for more things
um but it is a very privileged position to be able to do research uh where you know we basically get to
spend our days trying to figure things out you know with all the admin and all the academia things
and all the sort of all the stuff that goes on but at the end of the day i do get to spend a large
chunk of my time figuring things out and that is really really nice so the epistemic and the pragmatic
value tend to overlap which is nice well thank you it's a beautiful sentiment so it was a great series
this is really um a mega tech tree direction for the act in the ecosystem and beyond
so good luck with your continued work and play and research magnus it was awesome thanks i had a
blast coming on here it's been some really really great discussions and for those of you
participate in chat um got some hard questions that's good yes and it will continue in the institute discord
excellent i'll see you there see you magnus bye
is